**SECTION B NO. 1. CNN PROBLEM**

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import glob
import cv2
import os

import warnings
warnings.filterwarnings('ignore')

from sklearn.metrics import confusion_matrix
from keras.utils import to_categorical
from keras.models import Sequential
from keras.layers import Dense,Dropout,Flatten,Conv2D,MaxPool2D, AveragePooling2D
from keras.optimizers import Adam
from sklearn.metrics import accuracy_score
```

**PREPROCESSING STEP**

```
np.random.seed(1234)
directory="../DATASET/No.1/Fruit/Training"
classes=["Watermelon","Tomato", "Plum","Mango Red", "Banana"]
num_classes=len(classes)
all_arrays=[]
img_size=100
train_size=100
t=0
for i in classes:
    path=os.path.join(directory,i)
    class_num=classes.index(i)
    for img in os.listdir(path):
        # img_array=cv2.imread(os.path.join(path,img),
        #                      cv2.IMREAD_GRAYSCALE)
        img_array=cv2.imread(os.path.join(path,img))
        img_array=cv2.cvtColor(img_array, cv2.COLOR_BGR2RGB)
        img_array=cv2.resize(img_array,(img_size,img_size))
        all_arrays.append([img_array,class_num])
```

```
directory2="../DATASET/No.1/Fruit/Testing"
classes2=classes

all_arrays2=[]
img_size=100
for i in classes2:
    path=os.path.join(directory2,i)
    class_num2=classes2.index(i)
    for img in os.listdir(path):
        # img_array2=cv2.imread(os.path.join(path,img),
        #                      cv2.IMREAD_GRAYSCALE)
        img_array2=cv2.imread(os.path.join(path,img))
        img_array2=cv2.cvtColor(img_array2, cv2.COLOR_BGR2RGB)
        img_array2=cv2.resize(img_array2,(img_size,img_size))
        all_arrays2.append([img_array2,class_num2])
```

```
import random
random.shuffle(all_arrays)

X_train=[]
Y_train=[]
for features,label in all_arrays:
    X_train.append(features)
    Y_train.append(label)
X_train=np.array(X_train) #arraying

import random
random.shuffle(all_arrays2)

X_test=[]
Y_test=[]
for features,label in all_arrays2:
    X_test.append(features)
```

```
    Y_test.append(label)
X_test=np.array(X_test) #arraying
```

```
#normalization and reshaping
X_train=X_train.reshape(-1,img_size,img_size,3)
X_train=X_train/255
X_test=X_test.reshape(-1,img_size,img_size,3)
X_test=X_test/255
print("shape of X_train= ",X_train.shape)
print("shape of X_test=  ",X_test.shape)
```

```
from keras.utils import to_categorical
Y_train = to_categorical(Y_train, num_classes=num_classes)
Y_test = to_categorical(Y_test, num_classes=num_classes)
```

**a. We only use some parts of the data. Split your data into x_train,x_val,y_train,y_val using train_test_split with the test_size=30 and train_size=70**

```
from sklearn.model_selection import train_test_split

x_train, x_val, y_train, y_val = train_test_split(
    X_train, Y_train, test_size=30, train_size=70, random_state=1234
)

print("x_train shape:", x_train.shape)
print("x_val shape:", x_val.shape)
```

**b. Build your CNN model, compile and train it. Don't forget to store your `model.fit()` in `history` variable for plotting later (`history=model.fit(...)`).**

```
model = Sequential()

model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(img_size, img_size, 3)))
model.add(MaxPool2D(pool_size=(2, 2)))
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(MaxPool2D(pool_size=(2, 2)))
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(num_classes, activation='softmax'))

model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

history = model.fit(
    x_train, y_train,
    validation_data=(x_val, y_val),
    epochs=10,
    batch_size=16
)
```

**c. Draw your architecture model and submit it as separate .png file.**

```
from keras.utils import plot_model

plot_model(
    model,
    to_file='model_architecture.png',
    show_shapes=True,
    show_layer_names=True
)

model.summary()
```

**d. Plot the training and validation accuracy and loss**

```
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

plt.tight_layout()
plt.show()
```

**e. Print your accuracy score on test dataset provided in the testing folder**

```
Y_pred = model.predict(X_test)
Y_pred_classes = np.argmax(Y_pred, axis=1)
Y_true_classes = np.argmax(Y_test, axis=1)

test_accuracy = accuracy_score(Y_true_classes, Y_pred_classes)
print("Test Accuracy:", test_accuracy)
```